SESSION 4

Software Factories + Retro

From individual agent usage to repeatable team delivery systems

2–3 hours | Advanced

From Agents to Factories

Without factories, autonomous agents are just doing expensive vibe coding at scale.

S1 Vibe Coding

Fast prototypes,
unpredictable quality

→

S2 Discipline

Specs, TDD,
auditable process

→

S3 Agents

Autonomous
tool-using loops

→

S4 Factories

Agents + discipline
at team scale

Without factories:

- Inconsistent output across runs
- No audit trail
- Prompt drift
- Expensive compute, no quality guarantees

What Is an AI Software Factory?

An orchestrated pipeline where AI agents autonomously handle most of the SDLC — coding, testing, debugging, review, docs — while humans provide direction, architecture, and validation.

AI-Assisted Dev

Sessions 1-2

Human drives,
AI helps

Autonomous Agent

Session 3

Single agent
with tools

AI Software Factory

Session 4

Multiple agents
in governed pipeline

The human role shifts: from writing code to directing architecture, defining acceptance criteria, and validating outputs.

✓ Consistent quality ✓ Faster cycles ✓ Auditable traces ✓ Easy onboarding ✓ Scalable

Factory Architecture — 7 Layers

7.	Integration	CI/CD, repos, issue trackers
6.	Observability	Logs, traces, cost tracking
5.	Validation & Guardrails	Tests, hooks, approval gates
4.	Memory & Context	RAG, session state, learnings
3.	Tools & Execution	CLI, MCP, APIs, sandboxes
2.	Agent Layer	Planner, Builder, Reviewer...
1.	Orchestration	Spec intake, routing, loops

Maps to Training

Layers 2-5	Sessions 1-3
Layers 1, 6, 7	Session 4 (NEW)

**S1-S3 built layers 2-5.
S4 adds orchestration, observability & integration.**

AI Factories in the Wild

Company	What They Built	Key Insight
Stripe	"Minions" — Slack-triggered agents → PRs	Meet devs where they work
StrongDM	AI handles all coding, humans = roadmaps only	Human role = direction + validation
Cursor	Automations — background agents for tasks	IDE-native factory pattern
Factory.ai	Dedicated AI factory platform ($1.5B)	Factory-as-a-service is a market

Productivity Data

30-70% productivity gains

8-20× on migrations

~50% AI-generated code

REHEARSAL 1: Minimal Factory

Goal: Show the smallest possible factory shape — one agent, one call, structured I/O

```
uv run python 01_minimal_factory.py --backend claude "Summarize this project"
```

```
uv run python 01_minimal_factory.py --backend codex "Summarize this project"
```

```
uv run python 01_minimal_factory.py --backend opencode "Summarize this project"
```

Discussion:

1. What's the difference between calling an LLM vs calling a factory?
2. How does the backend choice affect the result?
3. Where would you add quality gates?

Factory Roles & Policies

Same backend, different policy → different behavior

Role	Permissions	Output	Constraints
Analyzer	Read-only	Findings report	No modifications
Planner	Read-only	Step-by-step plan	No execution
Fixer	Read + write targeted	Minimal diff	Scoped changes only
Reviewer	Read-only	Pass/fail + rationale	Against defined criteria

The core factory lever: control agent behavior through policy, not prompt engineering.

REHEARSAL 2: Role-Based + Resume

Three Roles, Same Backend

```
02_factory_catalog.py analyzer "Analyze backend_runner.py"

02_factory_catalog.py planner "Plan logging for backend_runner.py"

02_factory_catalog.py fixer "Add logging to backend_runner.py"
```

Session Resume

```
start "Remember: magic
       number is 42"

resume "What was the
        magic number?"
```

Role	Allowed Tools	Mode	Purpose
analyzer	Read, Glob, Grep	Read-only	Inspect and explain
planner	Read, Glob, Grep	Plan-only	Produce plans
fixer	Read, Grep, Edit, Bash	Accept edits	Make safe changes

Spec-Driven Delivery Loops

Read Spec → Implement → Run Checks → Review

↻ repeat until approved or max iterations

Spec Field	Description
goal	Target outcome
checks	Deterministic local commands (tests, lints)
max_iterations	Safety limit
implement_instructions	How to build (optional)
review_instructions	How to evaluate (optional)
allowed_tools	Tool whitelist (optional)

REHEARSAL 3: Spec Loop Factory

```
04_spec_loop_factory.py --backend claude --spec spec.example.json
```

```
04_spec_loop_factory.py --backend claude --spec spec.add-timeout.json
```

spec.add-timeout.json

```
{ "goal": "Add 120s timeout",

  "max_iterations": 3,

  "checks": [
    {"name": "syntax",
     "command": "{python} -m py_compile ..."},
    {"name": "timeout-exists",
     "command": "grep -q wait_for ..."}
  ]
}
```

Discussion

1. How does the loop decide when to stop?

2. What happens when checks fail?

3. Where does human review fit in?

What Can Go Wrong

Anti-Pattern	What Happens	Mitigation
Slop at Scale	High volume, low quality passes checks	Meaningful tests, human review gates
Prompt Drift	Team customizes independently, behavior diverges	Version-controlled role policies
Bag-of-Agents	Agents duplicate work or conflict	Single orchestration layer, handoff contracts
3 AM Liability	Overnight deploys, nobody reviews	Approval gates, observability alerts
Missing Observability	Nobody knows cost, decisions, or outcomes	Structured logging, decision traces

Every productivity tool can become a liability multiplier. Factories need production-grade operational discipline.

Session 4 Summary

1. Understood why agents need factory structure (vibe coding at scale)
2. Learned the 7-layer factory architecture
3. Seen real-world AI factory deployments and productivity data
4. Used role-based policies to control agent behavior
5. Built a spec-driven delivery loop
6. Identified common anti-patterns and mitigations
7. Completed a structured retro for sessions 1-3

Durable AI advantage comes from systems, not hero prompts. Factories turn agent capability into repeatable team delivery.

The Journey: Sessions 1-4

Session	Theme	Key Concept	You Built
S1: Greenfield	Proto Capability	Rules → Skills → Spec → TDD	Working prototype with TDD
S2: Brownfield	Hard Engineering	Document → Spec → Develop → Audit	Grounded spec + sub-agents
S3: Agents	Agent Engineering	Agent loop + CLI/MCP + RAG + memory	Custom agent with tools
S4: Factories	Factory & Team	7-layer arch + spec loops + policies	Repeatable factory pipeline

Session 1
Vibe Coding

→

Session 2
Spec-Driven Dev

→

Session 3
Agent Engineering

→

Session 4
Software Factories

Retro - Sessions 1-4

Retro questions

1. What did I learn during these four sessions?

2. Did I apply some of the learnings in my own daily work?

3. Was there something that I disagree with or could be better done otherwise?

4. What would I want to learn next?

Gather

Top 5 learnings

What are we excited about?

Feedback to Pasi

15 minutes — structured discussion, capture everything